Registering an array to StarPU from Fortran

```
1  real(8), dimension(:), allocatable, target :: array
2  allocate(array(NX))
3  array = (/ (1.0, i=1, NX) /)
4
5  type(c_ptr) :: vector_handle
6  call fstarpu_vector_data_register(vector_handle, &
7    STARPU_MAIN_RAM, c_loc(array), NX, c_sizeof(array(0)))
```

Declaring a codelet in Fortran

- Defined as a C_PTR allocated by StarPU

```
1 TYPE(C_PTR) :: scal_cl = C_NULL_PTR
2
3 scal_cl = fstarpu_codelet_allocate()
```

Declaring a codelet in Fortran

- Defined as a C_PTR allocated by StarPU
- Plug the kernel function

```
1 TYPE(C_PTR) :: scal_cl = C_NULL_PTR
2
3 scal_cl = fstarpu_codelet_allocate()
4
5 CALL fstarpu_codelet_add_cpu_func(scal_cl,
6   C_FUNLOC(scal_cpu_func))
```

Declaring a codelet in Fortran

- Defined as a C_PTR allocated by StarPU
- Plug the kernel function
- Declare data

```
1 TYPE(C_PTR) :: scal_cl = C_NULL_PTR
2
3 scal_cl = fstarpu_codelet_allocate()
4
5 CALL fstarpu_codelet_add_cpu_func(scal_cl,
6   C_FUNLOC(scal_cpu_func))
7
8 CALL fstarpu_codelet_add_buff(scal_cl, FSTARPU_RW)
```

Writing a Kernel Function in Fortran

- Every kernel function has the same Fortran prototype

```
1 recursive subroutine scal_cpu_func(buffers, cl_args) bind(c)
2   type(c_ptr), value, intent(in) :: buffers, cl_args
3
4   ...
```

Writing a Kernel Function in Fortran

- Every kernel function has the same Fortran prototype
- Get vector's number of elements and base pointer

```
1  recursive subroutine scal_cpu_func(buffers, cl_args) bind(c)
2    type(c_ptr), value, intent(in) :: buffers, cl_args
3
4    integer :: n
5    real(8), dimension(:), pointer :: val
6
7
8    n = fstarpu_vector_get_nx(buffers, 0)
9    call c_f_pointer(fstarpu_vector_get_ptr(buffers, 0), &
10     val, shape=[n])
11
12   ...
```

Writing a Kernel Function in Fortran

- Every kernel function has the same Fortran prototype
- Get vector's number of elements and base pointer
- Get the scaling factor as inline argument

```
1  recursive subroutine scal_cpu_func(buffers, cl_args) bind(c)
2    type(c_ptr), value, intent(in) :: buffers, cl_args
3
4    integer :: n
5    real(8), dimension(:), pointer :: val
6    real(8), target :: factor
7
8    n = fstarpu_vector_get_nx(buffers, 0)
9    call c_f_pointer(fstarpu_vector_get_ptr(buffers, 0), &
10     val, shape=[n])
11   call fstarpu_unpack_arg(cl_args, (/ c_loc(factor) /)))
12
13   ...
```

Writing a Kernel Function in Fortran

- Every kernel function has the same Fortran prototype
- Get vector's number of elements and base pointer
- Get the scaling factor as inline argument
- Compute the vector scaling

```
1  recursive subroutine scal_cpu_func(buffers, cl_args) bind(c)
2    type(c_ptr), value, intent(in) :: buffers, cl_args
3
4    integer :: n
5    real(8), dimension(:), pointer :: val
6    real(8), target :: factor
7
8    n = fstarpu_vector_get_nx(buffers, 0)
9    call c_f_pointer(fstarpu_vector_get_ptr(buffers, 0), &
10     val, shape=[n])
11   call fstarpu_unpack_arg(cl_args, (/ c_loc(factor) /)))
12
13   do i = 1, n
14     val(i) = val(i)*factor
15   end do
```

Submitting a task from Fortran

```
1   real(kind=c_double), target :: factor
2
3   call fstarpu_task_insert((/ scal_cl, &
4     FSTARPU_RW, vector_handle, &
5     FSTARPU_VALUE, c_loc(factor), FSTARPU_SZ_C_DOUBLE, &
6     C_NULL_PTR /))
```